

Relatório de Projeto

GREENHERB

Plataforma Web de Gestão Inteligente de uma Estufa

REALIZADO POR

Nome do Aluno 1 N.º 99999
Nome do Aluno 2 N.º 99999
Nome do Aluno 3 N.º 99999

Licenciatura em ...

Unidade Curricular de ...

Docente: Nome do/a Docente

2025/2026 · Semestre 2

6 de maio de 2026

Resumo

[Substituir pelo texto do resumo. Apresentar em poucas linhas o contexto do projeto GREENHERB, os principais requisitos abordados, a arquitetura geral adotada (browser, API, base de dados) e os resultados relevantes. O leitor que apenas leia este resumo deve ficar com uma ideia clara do que foi feito e do que se demonstrou.]

Palavras-chave: aplicação web, gestão de estufa, Node.js, MongoDB, armazenamento no browser, JWT, OpenAPI.

Conteúdo

Resumo	i
1 Introdução	1
1.1 Enquadramento	1
1.2 Objetivos	1
1.3 Contexto e restrições	1
1.4 Estrutura do documento	2
2 Requisitos	3
2.1 Atores do sistema	3
2.2 Regras de negócio	3
2.3 Requisitos funcionais	4
2.3.1 Detalhe e critérios de aceitação (exemplos)	4
2.4 Requisitos não funcionais	5
3 Arquitetura	6
3.1 Visão geral	6
3.2 Matriz de responsabilidades	6
3.3 Camada de cliente (<i>browser</i>)	7
3.3.1 Mecanismos de armazenamento	8
3.3.2 Decisão sobre o armazenamento do JWT	8
3.3.3 Estratégia de sincronização	8
3.4 Camada de servidor (API)	9
3.4.1 Organização das rotas	9
3.4.2 Cadeia de <i>middleware</i>	9
3.4.3 Motor de regras e modo Automático	10
3.5 Modelo de dados	10
3.6 Segurança	10
3.7 Resumo das decisões	11
4 Conclusão	12
4.1 Síntese do trabalho realizado	12
4.2 Avaliação dos critérios de aceitação	12
4.3 Reflexão crítica	12
4.3.1 O que correu bem	12
4.3.2 O que faríamos diferente	13
4.3.3 Aprendizagens da equipa	13
4.4 Trabalho futuro	13

A	Especificação OpenAPI	14
B	Dados de demonstração	15
C	Distribuição do trabalho	16

Lista de Figuras

3.1	Visão de alto nível da arquitetura em três camadas.	6
3.2	Modelo conceptual de dados (entidades e relações).	10

Lista de Tabelas

2.1	Perfis de utilizador e respetivas responsabilidades.	3
2.2	Lista priorizada de requisitos funcionais.	4
2.3	Requisitos não funcionais.	5
3.1	Distribuição de responsabilidades por camada.	7
3.2	Mecanismos de armazenamento no <i>browser</i> e respetivos propósitos.	8
3.3	Resumo dos principais <i>endpoints</i> da API.	9
3.4	Síntese das decisões de arquitetura.	11
4.1	Verificação dos critérios de aceitação do projeto.	12
C.1	Contribuição de cada elemento da equipa.	16

1

Introdução

1.1 Enquadramento

A empresa GREENHERB pretende informatizar a gestão de uma estufa dedicada à produção de ervas aromáticas, designadamente manjerição, hortelã, alecrim e tomilho. O sistema atual, maioritariamente manual, cria dificuldades no planeamento, na monitorização das condições ambientais e na rastreabilidade das intervenções realizadas.

[Completar: descrever o impacto destas dificuldades na operação quotidiana da estufa e a oportunidade que a digitalização representa.]

1.2 Objetivos

O presente trabalho tem como objetivos:

1. Desenvolver uma aplicação web *full-stack* que suporte o ciclo de vida completo de lotes de cultivo;
2. Implementar autenticação, autorização baseada em perfis e rastreabilidade das ações realizadas;
3. Especificar e documentar a API segundo a norma OpenAPI 3.x;
4. Definir uma arquitetura de armazenamento no browser que permita o funcionamento da aplicação em condições de conectividade instável;
5. [Adicionar / ajustar conforme decisões da equipa.]

1.3 Contexto e restrições

A solução é desenvolvida com restrições técnicas previamente definidas: o *frontend* em HTML, CSS e JavaScript, o *backend* em Node.js com recurso a Mongoose para acesso a uma base de dados MongoDB, autenticação baseada em JWT e uso obrigatório dos mecanismos de armazenamento do navegador (`localStorage` ou `sessionStorage`, IndexedDB e Cache API).

Acresce o contexto operacional: a aplicação é utilizada em ambiente de estufa onde a conectividade pode ser instável, o que condiciona fortemente as decisões de arquitetura do

lado do cliente.

1.4 Estrutura do documento

O restante documento organiza-se da seguinte forma. O Capítulo 2 identifica e especifica os requisitos do sistema, distinguindo entre funcionais e não funcionais e apresentando a sua priorização. O Capítulo 3 descreve a arquitetura adotada, detalhando as responsabilidades de cada camada (*browser*, API e base de dados) e justificando as decisões tomadas. O Capítulo 4 apresenta as conclusões, balanço crítico e trabalho futuro.

2

Requisitos

2.1 Atores do sistema

O sistema considera três perfis de utilizador, cada um com um âmbito distinto de operações.

Tabela 2.1: Perfis de utilizador e respetivas responsabilidades.

Perfil	Responsabilidades principais
Técnico	Registo de medições e tarefas operacionais; consulta de planos e lotes atribuídos.
Responsável	Definição de planos de cultivo; autorização de intervenções pontuais; resolução de alertas.
Administrador	Gestão de utilizadores e configuração geral do sistema; acesso integral aos dados.

2.2 Regras de negócio

As regras de negócio que condicionam o comportamento do sistema estão sintetizadas a seguir.

- RN-01** Cada plano de cultivo está associado a uma erva aromática e classifica-se como *regular*, *emergência* ou *pontual*.
- RN-02** Um plano *regular* inclui obrigatoriamente intervalos de temperatura, humidade e luminosidade, plano de rega e fertilização, e duração prevista do ciclo.
- RN-03** Um plano *pontual* exige autorização explícita do Responsável.
- RN-04** Os alertas classificam-se como *Informativo*, *Aviso* ou *Crítico* e podem ser *Resolvidos* ou *Ignorados*; a opção *Ignorado* exige justificação.
- RN-05** O sistema opera em modo *Manual* ou *Automático*; em modo automático pode sugerir ou executar ações com base em regras.
- RN-06** [Adicionar as restantes regras relevantes derivadas do enunciado.]

2.3 Requisitos funcionais

Os requisitos funcionais identificados estão listados na Tabela 2.2. A prioridade segue a notação MoSCoW: **M** (*must*, obrigatório), **S** (*should*, importante), **C** (*could*, desejável), **W** (*won't*, fora do âmbito desta versão).

Tabela 2.2: Lista priorizada de requisitos funcionais.

ID	Descrição	Atores	Prio.
RF-01	Autenticação de utilizadores e controlo de acesso por perfil.	Todos	M
RF-02	Gestão de utilizadores (criar, editar, desativar) pelo Administrador.	Admin.	M
RF-03	Registo, edição, consulta e importação (CSV/Excel) de ervas aromáticas.	Resp.	M
RF-04	Criação e gestão de planos de cultivo (<i>regular, emergência, pontual</i>).	Resp.	M
RF-05	Associação de planos a lotes e consulta do estado atual de cada lote.	Resp., Téc.	M
RF-06	Registo de tarefas operacionais e marcação como executadas.	Téc.	M
RF-07	Registo de medições ambientais (temperatura, humidade, luminosidade).	Téc.	M
RF-08	Geração automática de alertas quando os valores violam o plano.	Sistema	M
RF-09	Resolução ou <i>ignore</i> de alertas, com justificação obrigatória quando aplicável.	Resp.	M
RF-10	Suporte aos modos de operação <i>Manual</i> e <i>Automático</i> .	Resp.	S
RF-11	Histórico de medições, intervenções e alertas.	Todos	S
RF-12	Comparação entre valores reais e valores esperados pelo plano.	Resp.	S
RF-13	Exportação de relatórios em formato CSV ou Excel.	Resp., Admin.	S
RF-14	Divisão de lotes, registo de perdas e cálculo de produtividade.	Resp.	S
RF-15	Registo de logs de auditoria sobre as operações relevantes.	Sistema	M

2.3.1 Detalhe e critérios de aceitação (exemplos)

A seguir detalham-se dois requisitos a título de exemplo. Recomenda-se que esta especificação seja replicada para todos os requisitos classificados como **M**.

RF-08 — Geração automática de alertas. Quando uma medição ambiental é registada e algum dos seus valores está fora do intervalo definido no plano de cultivo associado ao lote, o sistema deve criar um alerta com a classificação adequada (*Informativo, Aviso* ou *Crítico*).

Critérios de aceitação:

- Dada uma medição cujo valor de temperatura excede em mais de 5 %°C o limite superior do plano, é gerado um alerta *Crítico* no prazo de 5 segundos.
- Dado um sensor que não envia dados há mais de 10 minutos, é gerado um alerta de *Aviso* associado ao lote.
- Os alertas gerados ficam visíveis no dashboard do Responsável e no histórico do lote.

RF-09 — Resolução ou *ignore* de alertas. O Responsável deve poder marcar um alerta como *Resolvido* ou *Ignorado*. No caso de *ignore*, é obrigatório fornecer uma justificação textual.

Critérios de aceitação:

- A interface impede submeter o *ignore* sem justificação.
- A decisão é registada no log de auditoria com identificação do utilizador, momento e justificação.
- Um alerta resolvido ou ignorado deixa de aparecer na lista ativa, mas mantém-se no histórico.

2.4 Requisitos não funcionais

Tabela 2.3: Requisitos não funcionais.

ID	Categoria	Descrição
RNF-01	Segurança	Palavras-passe armazenadas com <i>hashing</i> (e.g., bcrypt); nunca em texto claro.
RNF-02	Segurança	Comunicações cliente-servidor protegidas por HTTPS em produção.
RNF-03	Segurança	Autenticação via JWT, com expiração explícita e proteção das rotas.
RNF-04	Disponibilidade	A aplicação deve permitir registo de medições e tarefas em modo degradado (sem rede).
RNF-05	Usabilidade	Interface funcional em <i>desktop</i> e <i>mobile</i> (<i>responsive</i>).
RNF-06	Manutenibilidade	API documentada em OpenAPI 3.x e alinhada com a implementação.
RNF-07	Auditabilidade	Todas as operações sensíveis registam autor, momento e ação.
RNF-08	[Adicionar]	[Outros requisitos relevantes: desempenho, internacionalização, etc.]

3

Arquitetura

3.1 Visão geral

A solução adopta uma arquitetura em três camadas, separando claramente as responsabilidades do cliente, do servidor e da persistência. A Figura 3.1 apresenta a visão de alto nível.

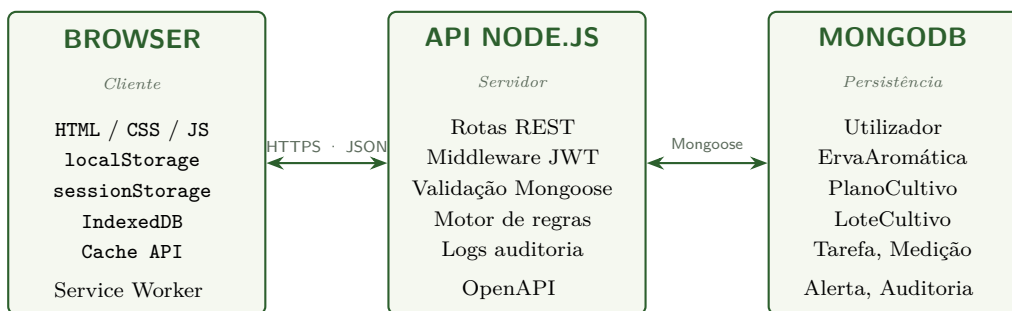


Figura 3.1: Visão de alto nível da arquitetura em três camadas.

A comunicação entre o cliente e o servidor faz-se sobre HTTPS, trocando mensagens em JSON. O servidor acede à base de dados através do ODM *Mongoose*, que assegura a validação de esquemas antes da persistência.

3.2 Matriz de responsabilidades

A Tabela 3.1 apresenta a distribuição de responsabilidades por camada para as funcionalidades-chave do sistema.

Tabela 3.1: Distribuição de responsabilidades por camada.

Funcionalidade	Browser	API	Base de Dados
Autenticação JWT	Envio de credenciais; armazenamento e renovação do token.	Validação de credenciais; emissão e verificação do JWT.	Persistência de utilizadores e <i>hashes</i> de palavra-passe.
Registo de medição ambiental	Captura via formulário; <i>enqueue</i> em IndexedDB se <i>offline</i> .	Validação de regras; persistência; verificação de limites.	Persistência da medição e dos alertas eventualmente gerados.
Geração de alertas	Apresentação ao utilizador; UX de resolução.	Avaliação dos limites do plano; criação dos alertas.	Persistência do alerta com referência ao lote e ao plano.
Importação de CSV/Excel	Pré-validação leve do ficheiro (formato, colunas).	Validação completa, normalização e inserção em <i>bulk</i> .	Persistência das ervas aromáticas importadas.
Modo Manual / Automático	UI de comutação e de listagem das ações sugeridas.	Motor de regras; execução ou sugestão das ações.	Registo da configuração do modo e do histórico de ações.
Sincronização <i>offline</i>	Fila de operações pendentes em IndexedDB; reenvio ao reconectar.	Tratamento idempotente das operações; resolução de conflitos.	Persistência idempotente; carimbos temporais.

3.3 Camada de cliente (*browser*)

A camada de cliente foi desenhada para suportar três cenários distintos de operação: ligação estável, ligação intermitente e ausência total de conectividade. Esta secção descreve os mecanismos de armazenamento utilizados, a respetiva justificação, política de invalidação e estratégia de sincronização.

3.3.1 Mecanismos de armazenamento

Tabela 3.2: Mecanismos de armazenamento no *browser* e respectivos propósitos.

Mecanismo	Dados guardados	Justificação	Expiração
<code>localStorage</code>	Preferências da UI; identificador do último ecrã visado; <i>feature flags</i> .	Pequenos pares chave/valor síncronos; persistem entre sessões.	Manual; limpa no <i>logout</i> .
<code>sessionStorage</code>	Estado temporário de formulários multi-passo (<i>wizards</i>).	Volátil por aba; evita “poluir” o estado entre sessões.	Fim da sessão da aba.
<code>IndexedDB</code>	Histórico recente; cache de lotes e medições; fila de operações pendentes (<i>offline</i>).	Estrutura, queryable, assíncrono; permite operação degradada.	TTL de [N] dias para o cache; a fila é purgada após sincronização bem-sucedida.
Cache API	Recursos estáticos (HTML, CSS, JS, ícones); respostas GET idempotentes selecionadas.	Gerida pelo Service Worker; reduz latência e suporta arranque <i>offline</i> .	Estratégia <i>stale-while-revalidate</i> ; invalidação por versão.

3.3.2 Decisão sobre o armazenamento do JWT

Decisão DA-01: armazenamento do token JWT

Decisão. O token JWT é guardado em [`localStorage` / `sessionStorage` / `cookie httpOnly`].

Alternativas consideradas.

- `localStorage`: persistência longa, mas exposto a ataques XSS.
- `sessionStorage`: limita a janela de exposição mas obriga a re-autenticar a cada sessão.
- `Cookie httpOnly + Secure + SameSite`: imune a XSS por JavaScript mas requer mitigação de CSRF.

Justificação. [Justificar com base no perfil de risco do sistema: estufa em ambiente operacional, sessões longas, baixa exposição a CSRF se a API não usa formulários tradicionais, etc.]

Mitigações. [Lista de medidas adotadas: *Content Security Policy* restritiva, *escape* de saídas, sanitização de *inputs*, expiração curta e renovação por *refresh token*, etc.]

3.3.3 Estratégia de sincronização

A operação em modo degradado segue o padrão *outbox*: as operações realizadas *offline* são enfileiradas em `IndexedDB` e reenviadas ao servidor assim que a conectividade é restabelecida. Para garantir a idempotência, cada operação carrega um identificador único gerado no cliente (UUID) que o servidor reconhece e ignora em caso de submissões repetidas.

Em caso de conflito (e.g., duas equipas registam a mesma colheita do mesmo lote em momentos diferentes), o sistema adota a estratégia de *[last-write-wins / merge / flag para revisão humana]*. Esta decisão tem implicações operacionais e deve ser discutida explicitamente.

3.4 Camada de servidor (API)

3.4.1 Organização das rotas

A API expõe os recursos da aplicação seguindo as convenções REST. A Tabela 3.3 sintetiza os principais *endpoints* agrupados por recurso. A especificação completa encontra-se em `openapi.yaml` (ver Anexo A).

Tabela 3.3: Resumo dos principais *endpoints* da API.

Método	Rota	Descrição
POST	<code>/auth/login</code>	Autenticação e emissão do JWT.
GET	<code>/utilizadores</code>	Listagem de utilizadores (Admin.).
POST	<code>/ervas/import</code>	Importação em <i>bulk</i> (CSV/Excel).
GET	<code>/lotes</code>	Listagem de lotes com estado atual.
POST	<code>/lotes/:id/medicoes</code>	Registo de medição ambiental.
PATCH	<code>/alertas/:id</code>	Resolver ou ignorar um alerta.

3.4.2 Cadeia de *middleware*

Cada pedido atravessa uma cadeia de *middleware* ordenada da seguinte forma:

1. **Logger HTTP:** regista método, rota, IP e *user-agent*.
2. **Validação JWT:** verifica assinatura e expiração; rejeita pedidos não autenticados em rotas protegidas.
3. **Autorização por perfil:** confronta o perfil do utilizador (RF-01) com os requisitos da rota.
4. **Validação de *payload*:** valida o corpo do pedido contra o esquema Mongoose ou contra um *schema* JSON.
5. **Controlador da rota:** executa a lógica de negócio.
6. ***Middleware* de erros:** converte exceções em respostas HTTP coerentes com a OpenAPI.

3.4.3 Motor de regras e modo Automático

Decisão DA-02: localização do motor de regras

Decisão. O motor de regras corre na API, e não no cliente.

Justificação. Embora o cliente possa fornecer *feedback* otimista quando uma medição é registada *offline*, a fonte da verdade tem de estar no servidor: (i) os limites do plano podem mudar; (ii) a coerência entre lotes exige uma visão global; (iii) no modo *Automático*, ações como ativar a rega ou ajustar a ventilação têm efeitos físicos que não podem depender do estado local do cliente.

3.5 Modelo de dados

A Figura 3.2 apresenta o modelo conceptual das entidades persistidas em MongoDB. Por simplicidade, são omitidos atributos secundários.

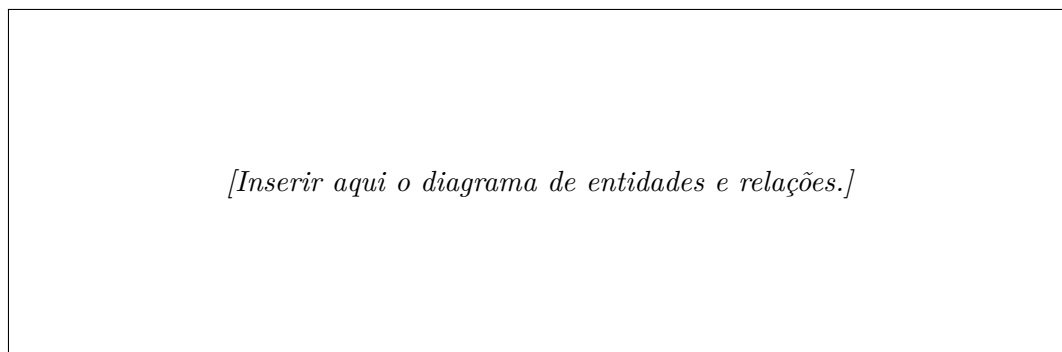


Figura 3.2: Modelo conceptual de dados (entidades e relações).

Em MongoDB, cada documento pode “embeber” subdocumentos ou referenciar outros documentos por `ObjectId`. As decisões de *embedding* versus *referencing* influenciam o desempenho das consultas e devem ser justificadas. Recomenda-se uma subsecção dedicada onde, para cada par de entidades relacionadas, se indique a opção tomada e o seu motivo.

3.6 Segurança

As principais decisões de segurança adotadas são as seguintes:

- As palavras-passe são armazenadas com *hashing* adaptativo (`bcrypt`, fator de custo ≥ 10).
- A autenticação usa JWT assinados (HS256 ou RS256) com *payload* mínimo e expiração de [N] minutos.
- Todas as rotas, exceto as públicas, exigem JWT válido.
- A autorização é controlada por perfil; o servidor nunca confia em campos sensíveis enviados pelo cliente (e.g., o perfil do utilizador é sempre obtido do JWT, não do corpo

do pedido).

- Os *inputs* são validados contra esquemas; as saídas que possam conter texto introduzido pelo utilizador são *escaped*.
- As ações sensíveis (criação de planos, autorização de planos pontuais, eliminação de utilizadores) ficam registadas em `LogAuditoria`.

3.7 Resumo das decisões

Tabela 3.4: Síntese das decisões de arquitetura.

ID	Decisão	Secção
DA-01	Armazenamento do JWT em [...]	3.3.2
DA-02	Motor de regras na API	3.4.3
DA-03	Sincronização <i>offline</i> via fila idempotente em IndexedDB	3.3.3
DA-04	[Adicionar conforme as decisões reais da equipa]	—

4

Conclusão

4.1 Síntese do trabalho realizado

[Apresentar de forma sintética o que foi efetivamente implementado. Recomenda-se referir a cobertura dos requisitos funcionais (RF-01 a RF-15) e dos requisitos não funcionais, indicando claramente o que ficou completo, parcial e por implementar.]

4.2 Avaliação dos critérios de aceitação

A Tabela 4.1 confronta o trabalho realizado com os critérios de aceitação definidos no enunciado.

Tabela 4.1: Verificação dos critérios de aceitação do projeto.

ID	Critério	Estado
CA-01	Um utilizador autenticado acede apenas às operações permitidas pelo seu perfil.	
CA-02	O sistema suporta os três tipos de plano de cultivo e a sua associação a lotes.	
CA-03	O registo de medições ambientais pode originar alertas automáticos.	
CA-04	O sistema demonstra comportamento distinto em modo <i>Manual</i> e <i>Automático</i> .	
CA-05	É possível consultar histórico, exportar relatórios e observar rastreabilidade.	
CA-06	A arquitetura de armazenamento no <i>browser</i> está implementada e coerente.	
CA-07	A implementação está alinhada com a especificação OpenAPI.	

Legenda: preencher cada linha com ✓ (cumprido), ~ (parcialmente cumprido) ou × (não cumprido), seguido de uma breve evidência.

4.3 Reflexão crítica

4.3.1 O que correu bem

[Identificar os aspetos do projeto onde a equipa considera ter tomado boas decisões. Justificar com evidências concretas — não basta dizer “a arquitetura ficou bem”; é preciso explicar porquê.]

4.3.2 O que faríamos diferente

[Identificar honestamente as decisões que, em retrospectiva, a equipa repensaria. Esta é uma das partes mais valorizadas do relatório: mostra capacidade de aprendizagem e pensamento crítico.]

4.3.3 Aprendizagens da equipa

[Aprendizagens técnicas e de processo. Que conceitos ficaram mais claros? Que ferramentas mostraram ser mais ou menos adequadas? Que prática de equipa funcionou melhor?]

4.4 Trabalho futuro

Identificam-se como direções relevantes para iterações futuras:

- [Listar funcionalidades não implementadas que fariam sentido — e.g., notificações *push*, *dashboards* avançados, integração com sensores reais.]
- [Listar melhorias de natureza técnica — e.g., *cobertura de testes automatizados*, *observabilidade*, *hardening de segurança*.]
- [Listar mudanças de processo ou metodologia que ajudariam num projeto semelhante.]



Especificação OpenAPI

A especificação completa da API está disponível no ficheiro `openapi.yaml` no repositório do projeto (<https://github.com/equipa/greenherb>). A título ilustrativo apresenta-se a seguir o excerto correspondente ao *endpoint* de autenticação.

```
paths:
  /auth/login:
    post:
      summary: Autenticar utilizador
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email: { type: string, format: email }
                password: { type: string, format: password }
      responses:
        "200":
          description: Autenticacao bem-sucedida
          content:
            application/json:
              schema:
                type: object
                properties:
                  token:
                    type: string
                    description: JWT assinado
        "401":
          description: Credenciais invalidas
```

Listing A.1: Excerto de `openapi.yaml` — autenticação.



Dados de demonstração

O dataset de demonstração inclui:

- [N] utilizadores (um por perfil, no mínimo).
- [N] ervas aromáticas, importadas a partir do ficheiro `ervas-exemplo.csv`.
- [N] planos de cultivo (*regular*, *emergência* e *pontual*).
- [N] lotes em diferentes estados (*ativo*, *concluído*, *comprometido*).
- Histórico de medições ambientais e alertas associados.

C

Distribuição do trabalho

Tabela C.1: Contribuição de cada elemento da equipa.

Elemento	Contribuições principais	%
Aluno 1	[Áreas de responsabilidade e principais entregas]	
Aluno 2	[Áreas de responsabilidade e principais entregas]	
Aluno 3	[Áreas de responsabilidade e principais entregas]	
Total		100%